PROPOSAL • OCTOBER 15, 2026

Agent Arena

An Autonomous Agent Benchmarking and Replay Platform

Team members	Aidan (A), Carter(B), Dakota(C)
GitHub repository	[Link to shared private course repository]
Proposal due	October 18, 2026

1. Problem, users, and proposed solution

AI-driven and rule-based software agents can complete multi-step web tasks, but evaluating their reliability is difficult. A successful-looking run may contain incorrect form values, unnecessary actions, or an unreported failure. Developers need a reproducible way to compare agent strategies using the same conditions and measurable outcomes.

Agent Arena is a web application where students and developers choose a controlled browser challenge, run two autonomous agent strategies against identical starting conditions, monitor their actions, and compare the results. The system records each action in a relational database, checks success against objective task requirements, and provides an evidence-based replay. The application is intended for students learning automation, software engineers, and developers testing agent behavior.

The demonstration environment will contain locally hosted tasks, such as scheduling an appointment or submitting a customer-support request. No real customer accounts, payment transactions, or third-party site automation are required.

2. Five integrated core features

1 Challenge selection and configuration

Users select a sample browser task, set permitted task inputs, and start an experiment. The application validates inputs and saves the challenge configuration.

Observable result: A saved, reproducible task with a clear target outcome and resettable initial state.

2 Two-agent execution

Users launch two strategies on the same task: a baseline agent following a fixed plan and an observation-aware agent that selects actions from the current page state. Each agent runs in an isolated, reset test environment.

Observable result: Two completed or failed runs with recorded actions, statuses, and outputs.

3 Live execution dashboard

Users watch action-by-action progress, including current step, status, action description, timestamps, and available screenshots. Updates stream from the backend to the interface.

Observable result: A changing dashboard that clearly shows what each agent is doing.

4 Objective scoring and diagnosis

Users compare performance. The evaluation module checks the submitted outcome against task ground truth, counts invalid actions, and reports completion, action count, and elapsed time. It identifies the first recorded failed step when possible.

Observable result: An auditable result and clear success/failure explanation for each agent.

5 History, replay, and comparison

Users open previous experiments, step through saved action traces and screenshots, and view side-by-side differences. They can rerun a challenge from a freshly reset state.

Observable result: Searchable historical results and a replay showing why runs differed.

3. Main components, tools, and integration

Component	Planned implementation / role
Web interface	React and TypeScript: challenge setup, live run monitor, score comparison, and replay timeline.
Application API	Python FastAPI: validates requests, coordinates runs, exposes history and scores.
Automation engine	Python Playwright: executes browser actions in a local test website with isolated scenarios.
Relational database	SQLite with SQLAlchemy and SQL: stores challenges, experiments, individual runs, actions, and evaluations.
Real-time updates	Server-Sent Events (SSE): delivers action status and progress from the backend to the browser.
Scripting / packaging	Python scripts seed demo scenarios, reset test data, and run checks; Docker Compose builds and runs services.

The database will use related tables including challenges (task definitions), experiments (a head-to-head comparison), runs (one agent attempt), run_actions (ordered tool calls and observations), and evaluations (outcomes and metrics). Foreign keys will connect each experiment to its challenge, each run to an experiment, and each action/evaluation to a run. Screenshots may be stored as linked files rather than database blobs.

The Python backend will apply object-oriented design through Challenge, AgentPolicy, BrowserRunner, TraceRecorder, and Evaluator components. Each class or module has a distinct responsibility: define goals, choose actions, execute them, store evidence, and score results. The two agent policies will implement a shared action-selection interface, making their comparison fair and their behavior testable.

4. Ambitious-project scope and controlled design

The ambitious element is the integration of independent automation policies, controlled browser execution, live telemetry, persistent action-level evidence, objective scoring, and replay within one application. The agents will be compared against the same task definition and reset starting data. Evaluation will use explicit ground-truth checks rather than an AI model deciding its own score.

The initial scope will support two or three locally hosted browser challenges. The baseline policy will execute a predetermined valid action plan; the observation-aware policy will inspect page elements and choose actions based on detected labels and state. The team will deliberately include a reproducible variant (for example, a renamed or reordered form field) to demonstrate different strategy outcomes. Browser execution is constrained to the local demonstration site.

Optional extension (not required for the five core features): add a language-model-backed agent policy that uses the same action interface and evaluation rules. This would require an externally configured API key; the normal demonstration will work without it. We will discuss bonus eligibility and the additional scope before undertaking extension work.

5. Development milestones and target dates

Milestone / date	Tasks and responsible students	Expected result / completion check
M1 — Foundation Nov. 5, 2026	Student A: API and database schema; Student B: React challenge forms; Student C: scenario website and seed/reset scripts. All review one another's work.	Save and retrieve a configured task; seed and reset demo data; start services locally; tests pass on valid and invalid inputs.
M2 — Live execution Nov. 16, 2026	Student A: Playwright runner and policies; Student B: SSE monitoring interface; Student C: run/action persistence and integration tests.	Both policies attempt one identical challenge; dashboard displays live actions; stored action histories match observed events.

M3 — Evaluation and delivery Nov. 24, 2026	Student A: objective scoring; Student B: replay/compare UI; Student C: Docker Compose, end-to-end checks, demo reset. All rehearse.	All five features demonstrated on two scenarios; fresh Docker build works; failure case and successful replay verified.

Milestone 1 finishes before the November 8 midterm report, and Milestone 3 finishes before presentation week (November 30 / December 2). The remaining time is reserved for bug fixes, documentation, and demo practice. Target dates may be revised with changes documented in the midterm and final reports.

6. Planned student responsibilities

Task / component	Expected result	Developer(s)	Tester / reviewer	Milestone
Database and API	Persistent challenges and run records	Student A	Student C	M1
Challenge UI	Task setup and validation	Student B	Student A	M1
Demo site and scripts	Repeatable local test scenarios	Student C	Student B	M1
Agent execution	Two policy strategies with action tracing	Student A	Student C	M2
Monitoring / replay	Live updates and timeline comparison	Student B	Student A	M2–M3
Evaluation, delivery	Ground-truth scoring, tests, Docker	Students A and C	Student B	M3

7. Verification, risk management, and live demonstration

Testing will verify each core feature and at least one invalid-input or boundary case. Planned checks include rejecting a blank or malformed task, ensuring identical initial data for both agents, verifying ordered run actions and database foreign-key relationships, confirming score calculations, and reconstructing a prior trace accurately. We will record test inputs, expected outcomes, actual results, pass/fail status, and tester. Results will only be reported after they are run.

The main risks are fragile browser selectors, timing variability, and integration across frontend, backend, and browser processes. To reduce risk, the application will use a local demo website, fixed scenarios, deterministic reset scripts, timeouts with clear failure statuses, and Docker-based smoke tests. Live execution and replay will work without a paid AI API or internet access.

During the in-class live demonstration, the team will: (1) choose a challenge; (2) start both agent policies; (3) show live action streaming; (4) display the objective results and an intentionally triggered failure; and (5) step through the recorded replay and compare results. Each student will explain the component they developed. A reset script will return the environment to a clean state before the demonstration.

8. GitHub, responsible AI use, and deliverables

The team will create a shared private GitHub repository and grant access to the instructor. The repository will include source code, SQL schema and sample data, setup/reset scripts, tests and test records, Docker files, a README with reproducible commands, presentation materials, and required PDF reports. Git history will show each student's meaningful implementation work throughout the milestones.

AI assistance may be used for planning, coding suggestions, and debugging. All project-related AI conversations and editor interactions will be preserved in a readable ai/ directory, with the student, tool, and date identified. Students will review outputs, verify accepted contributions, document limitations, and include the course-required responsibility statements in their individual reports.